TREASURE HUNT – JUNGLE RUIN

Unity 3D Game Design, Mechanics, and Development

A 3D adventure game in which the player explores an ancient jungle, overcomes obstacles, collects clues, and discovers a hidden treasure.

Project element	Design
Genre	3D adventure / exploration
Engine	Unity 3D
Player	Explorer character
Main goal	Find the hidden treasure in the ancient ruin
Collectibles	Coins, keys, and treasure clues
Challenges	Broken bridges, pits, traps, and locked doors
Completion	Unlock the treasure chamber and collect the treasure

1. Introduction

Treasure Hunt – Jungle Ruin is a 3D adventure game where the player takes the role of an explorer searching for a lost treasure hidden inside an ancient jungle temple. The player travels through forest paths, ruined stone structures, bridges, and underground chambers. Along the way, the player collects coins and keys, finds clues, and avoids environmental hazards.

Unity provides tools for building the environment, controlling the player, applying physics, detecting collisions, managing the camera, and displaying the score. C# scripts are used to connect these game systems.

Game objective: Explore the jungle, collect the required key items, avoid hazards, unlock the ancient chamber, and reach the treasure to complete the level.

2. Game Design and Mechanics

Game element	Design and mechanics
Player	3D explorer character controlled by the player
Environment	Jungle trees, vines, stone paths, temple ruins, caves
Movement	Move forward, backward, left, and right
Jumping	Use the Spacebar to jump across gaps and obstacles
Collectibles	Coins increase score; keys and clues help unlock the route
Obstacles	Pits, broken bridges, fallen logs, and trap zones
Camera	Follows the explorer from behind
Physics	Rigidbody, gravity, and collider-based interaction
Game Over	Triggered when the player falls into a pit or hazard
Winning	Triggered when the player reaches and collects the treasure

3. Step-by-Step Game Development Procedure

Step 1: Create a New Unity 3D Project

Open Unity Hub, select New Project, and choose a 3D template such as Universal 3D (URP). Name the project **TreasureHuntJungleRuin**. Create folders named Scenes, Scripts, Materials, Prefabs, and Models. Save the main scene as **JungleRuinLevel** inside the Scenes folder.

Step 2: Build the Jungle Environment

Create a terrain or a large ground plane. Add trees, bushes, rocks, vines, and ancient stone structures using Unity objects or suitable assets. Use green and brown materials for the jungle and weathered stone materials for the ruins. Add a path that guides the player from the jungle entrance toward the temple.

Step 3: Create the Player Character

Add a Capsule or humanoid model and rename it **Player**. Place it near the jungle entrance. Add a Rigidbody and Capsule Collider if they are not already present. Freeze Rigidbody rotation on the X and Z axes so the character does not tip over. Keep gravity enabled.

Step 4: Implement Player Movement and Jumping

Create a C# script named **PlayerController.cs** in the Scripts folder and attach it to the Player. This beginner-friendly example uses Unity's legacy Input Manager. If needed, set Active Input Handling to Input Manager (Old) or Both in Project Settings.

```
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class PlayerController : MonoBehaviour
{
    public float moveSpeed = 5f;
    public float jumpForce = 6f;
    private Rigidbody rb;
    private bool grounded;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
        rb.constraints = RigidbodyConstraints.FreezeRotation;
    }

    void Update()
    {
        if (Input.GetButtonDown("Jump") && grounded)
        {
            rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
            grounded = false;
        }
    }

    void FixedUpdate()
    {
        float x = Input.GetAxisRaw("Horizontal");
        float z = Input.GetAxisRaw("Vertical");
        Vector3 move = new Vector3(x, 0f, z).normalized * moveSpeed;
        rb.linearVelocity = new Vector3(move.x, rb.linearVelocity.y, move.z);
    }

    void OnCollisionStay(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
            grounded = true;
    }
}
```

Setup note: Tag walkable ground objects as Ground. In Unity versions that use Rigidbody.velocity instead of Rigidbody.linearVelocity, replace rb.linearVelocity with rb.velocity. Test the movement and jump before adding more objects.

Step 5: Add Obstacles and Routes

Build narrow paths, raised stone platforms, broken bridges, fallen trees, and safe jumping gaps. Place hazards in clearly marked areas. Make sure every required jump is reachable and that the player has a fair route through the

level.

4. Collectibles, Hazards, and Level Progress

Step 6: Add Coins, Keys, and Clues

Create small objects for coins and keys. Add a Collider to each collectible and enable **Is Trigger**. Give the player a Collider and a suitable tag. When the player touches a coin, increase the score and remove the coin. Keys or clues can be used to open a door to the treasure chamber.

```
using UnityEngine;

public class Collectible : MonoBehaviour
{
    public int points = 1;

    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            GameManager manager = FindObjectOfType<GameManager>();
            if (manager != null)
                manager.AddScore(points);

            Destroy(gameObject);
        }
    }
}
```

Attach this script to coin objects. For keys, use a separate key counter or inventory variable so the treasure door can check whether the required key has been collected.

Step 7: Add Jungle Hazards

Use trigger volumes or colliders for pits and trap areas. When the player enters a hazard, show a Game Over message and provide a restart option. Avoid placing hazards where the player cannot see or reasonably avoid them.

Step 8: Create the Treasure Door and Win Condition

Place an ancient stone door near the end of the level. The door should open only after the player collects the required key or clue. Put a treasure chest inside the final chamber. When the player reaches the chest, display a Victory message such as “Treasure Found!” and stop further gameplay or load a completion screen.

Step 9: Camera Follow Mechanics

Position the Main Camera behind and above the player. A simple follow script can keep a fixed offset while smoothing camera movement.

```
using UnityEngine;

public class CameraFollow : MonoBehaviour
{
    public Transform target;
    public Vector3 offset = new Vector3(0f, 4f, -6f);
    public float smoothSpeed = 5f;

    void LateUpdate()
    {
        if (target == null) return;

        Vector3 desired = target.position + offset;
        transform.position = Vector3.Lerp(
            transform.position, desired,
            smoothSpeed * Time.deltaTime);

        transform.LookAt(target);
    }
}
```

Attach the script to the Main Camera and drag the Player object into the Target field in the Inspector.

5. User Interface and Testing

Step 10: Add Score and Game Messages

Create a Canvas with Text or TextMeshPro elements to display the coin score, collected keys, instructions, and game status. Add simple messages such as “Find the key,” “Door unlocked,” “Treasure Found,” and “Game Over.” Include a restart button if time permits.

Suggested Level Layout

Area	Example placement / purpose
Jungle Entrance	Player spawn point and movement tutorial
Forest Path	Trees, rocks, and first coins
Broken Bridge	A short jumping challenge
Ancient Courtyard	Clues, key collectible, and stone pillars
Temple Entrance	Locked door that checks for the key
Treasure Chamber	Treasure chest and final win trigger

Testing Checklist

• Player can move in all intended directions and jump.

• Camera follows the player without losing sight of the route.

• Coins and keys disappear when collected and update the interface.

• Player can cross every required bridge and gap.

• Hazards trigger the Game Over state correctly.

• Locked door responds correctly to the key or clue.

• Treasure chest triggers the Victory state.

• Restarting the game resets the player and collectibles.

6. Expected Game Output

After successful implementation, Treasure Hunt – Jungle Ruin will provide a playable 3D adventure in which the explorer moves through a jungle environment, jumps over obstacles, collects coins and keys, and explores an ancient temple. The game will include camera follow, collision detection, a score display, hazard handling, a locked treasure chamber, and a clear victory condition.

7. Conclusion

Treasure Hunt – Jungle Ruin demonstrates core Unity game-development concepts, including 3D environment design, player movement, Rigidbody physics, collisions, triggers, collectibles, camera control, and game-state management. The project can be expanded with sound effects, animations, puzzles, multiple levels, moving traps, checkpoints, and a more detailed inventory system.

The main goal is to create an engaging exploration experience where the player uses movement and observation to overcome obstacles and discover the hidden treasure.